



Непрерывная разработка и интеграция CI/CD

...

Лекция 4

Волков Михаил Юрьевич

Тема 4

Принцип сборки и применения языка программирования

Система управления пакетами

Система управления пакетами — набор программного обеспечения, позволяющего управлять процессом установки, удаления, настройки и обновления различных компонентов программного обеспечения. Системы управления пакетами активно используются в различных дистрибутивах операционной системы Linux и других UNIX-подобных операционных системах.



Система управления пакетами



Автоматизация сборки

Автоматизация сборки – этап процесса разработки программного обеспечения, заключающийся в автоматизации широкого спектра задач, решаемых программистами в их повседневной деятельности.

Включает такие действия, как:

- Компиляция исходного кода в объектный модуль;
- Сборка бинарного кода в исполняемый файл;
- Выполнение тестов;
- Развёртывание программы в целевой среде;
- Написание сопроводительной документации или описание изменений новой версии.

Автоматизация сборки

Основное средство автоматизации сборки – применение специализированного инструмента; один из ранних и исторически значимых инструментов является утилита `make`, во многом определившая стиль и методы для инструментов, появившихся позднее. Один из таких элементов – формат `Makefile`, поддерживаемый в большинстве широко используемых инструментов (`Automake`, `CMake`, `imake`, `qmake`, `nmake`, `wmake`, `Apache Ant`, `Apache Maven`, `OpenMake Meister`, `Gradle`).



Автоматизация сборки

Виды автоматизации, применяемые в различных инструментах:

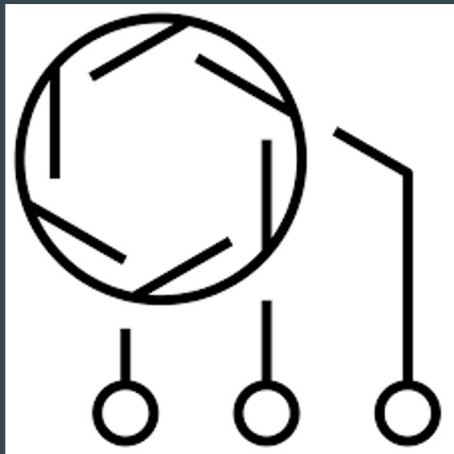
- автоматизация по запросу (on-demand automation): запуск пользователем сценария в командной строке,
- запланированная автоматизация (scheduled automation): непрерывная интеграция, происходящая в виде ночных сборок,
- условная автоматизация (triggered automation): непрерывная интеграция, выполняющая сборку при каждом подтверждении изменения кода (commit) в системе управления версиями.

Распределённая сборка

Распределённая сборка подразумевает, что вызовы компилятора и компоновщика могут передаваться множеству компьютеров для ускорения скорости сборки. Распределённый процесс сборки должен обладать определённой логикой, чтобы правильно определить зависимости в исходном коде для того чтобы выполнить этапы компиляции и компоновки на разных машинах.

Распределённая сборка

Решение автоматизации сборки должно быть способно управлять этими зависимостями, чтобы выполнять распределённые сборки. Некоторые инструменты сборки могут распознавать подобные взаимосвязи автоматически (Rational ClearMake distributed, Electric Cloud ElectricAccelerator), а другие зависят от пользовательских указаний (Platform LSF Ismake).



Организация процесса

Для организации процесса непрерывной интеграции на выделенном сервере запускается служба, в задачи которой входят:

- Получение исходного кода из репозитория;
- Сборка проекта;
- Выполнение тестов;
- Развёртывание готового проекта;
- Отправка отчетов.

Преимущества и недостатки

К преимуществам непрерывной интеграции относятся:

- Проблемы интеграции выявляются и исправляются быстро, что оказывается дешевле;
- Немедленный прогон модульных тестов для свежих изменений;
- Постоянное наличие текущей стабильной версии вместе с продуктами сборки — для тестирования, демонстрации;
- Немедленный эффект от неполного или неработающего кода приучает разработчиков к работе в итеративном режиме с более коротким циклом.

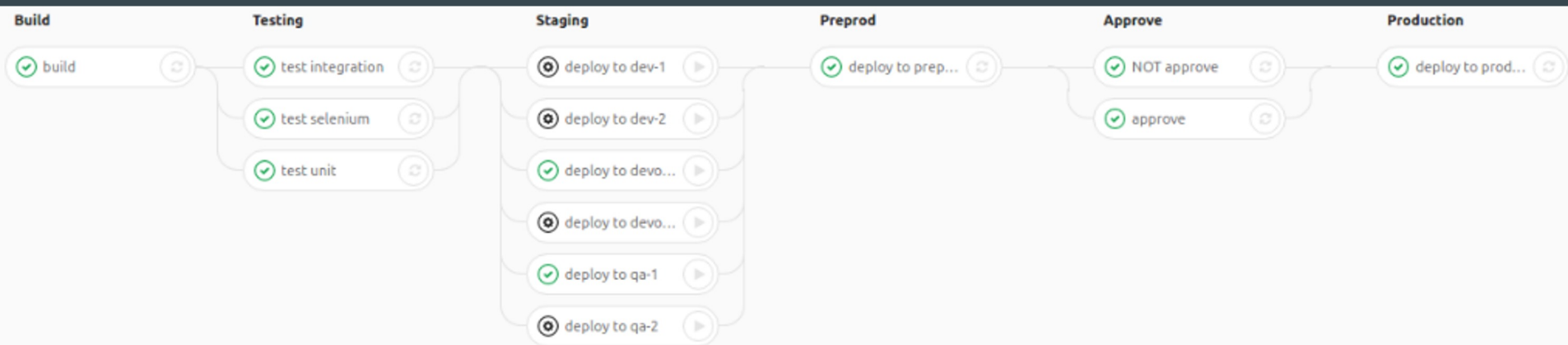
Преимущества и недостатки

К недостаткам непрерывной интеграции относятся:

- Значительные затраты на поддержку работы непрерывной интеграции;
- Необходимость в дополнительных вычислительных ресурсах под нужды непрерывной интеграции;
- Немедленный эффект от неполного или неработающего кода.

GitLab CI для непрерывной интеграции

Пайплайн— это последовательность выполнения стадий, каждая из которых включает несколько задач. От момента внесения изменений в код до production приложение по очереди оказывается в руках разных команд — подобному тому, как это происходит на конвейере.



Использование пайплайна

1. Разработчики выкладывают код приложения в ветки с префиксом `feature_`, а DevOps-инженеры — код инфраструктуры в ветки с префиксом `infra_`. Каждый `git push` в эти ветки запускает процесс сборки приложения (стадия `build`) и автоматические тесты (стадия `testing`).
2. Задачи на следующих стадиях не вызываются автоматически, а определены как задачи с ручным запуском (`manual`).
3. На стадии `staging` можно запустить задание и предоставить собранное и протестированное приложение на упрощенное окружение. Это могут делать разработчики, DevOps-инженеры, тестировщики.
4. После успешного прохождения тестов и представления на одно из окружений `staging` можно предоставить приложение в `pre-production` — это делают только тестировщики, DevOps-инженеры или релиз-инженеры.

Использование пайплайна

5. По мере накопления успешно протестированных функций релиз-инженер готовит новую версию и создаёт в репозитории тег. Пайплайн для тега отличается от пайплайна для ветки двумя дополнительными стадиями.
6. После успешной сборки, тестов и представления в pre-production проводятся дополнительные ручные тесты новой версии и показ заказчику приложения.
7. Представление приложения на production возможен только после успешного представления на pre-production и выполнения задания approve.

Пайплайн и стадии в деталях

1. Стадии build

2. Стадия staging

3. Стадия pre-production

4. Стадия approve

5. Стадия production

Разработка .gitlab-ci.yml

```
stages:
  - build
  - testing
  - staging
  - preprod
  - approve
  - production

## build stage
build:
  stage: build
  tags: [deploy]
  script:
    - echo "Build"

## testing stage
test unit:
  stage: testing
  tags: [deploy]
  script:
    - echo "test unit"

test integration:
  stage: testing
  tags: [deploy]
  script:
    - echo "test integration"

test selenium:
  stage: testing
  tags: [deploy]
  script:
    - echo "test selenium"

## staging stage
.staging-deploy: &staging-deploy
tags: [deploy]
```

```
stage: staging
when: manual
script:
  - echo $CI_BUILD_NAME

deploy to dev-1:
  <<: *staging-deploy

deploy to dev-2:
  <<: *staging-deploy

deploy to devops-1:
  <<: *staging-deploy

deploy to devops-2:
  <<: *staging-deploy

deploy to qa-1:
  <<: *staging-deploy

deploy to qa-2:
  <<: *staging-deploy

## preprod stage
deploy to preprod:
  stage: preprod
  tags: [deploy]
  when: manual
  script:
    - echo "deploy to preprod"

## approve stage
approve:
  stage: approve
```

```
tags: [deploy]
when: manual
script:
  - echo "APPROVED"
```

```
NOT approve:
  stage: approve
  tags: [deploy]
  when: manual
  script:
    - echo "NOT APPROVED"
```

```
## production stage
deploy to production:
  stage: production
  tags: [deploy]
  when: manual
  script:
    - echo "deploy to production!"
```

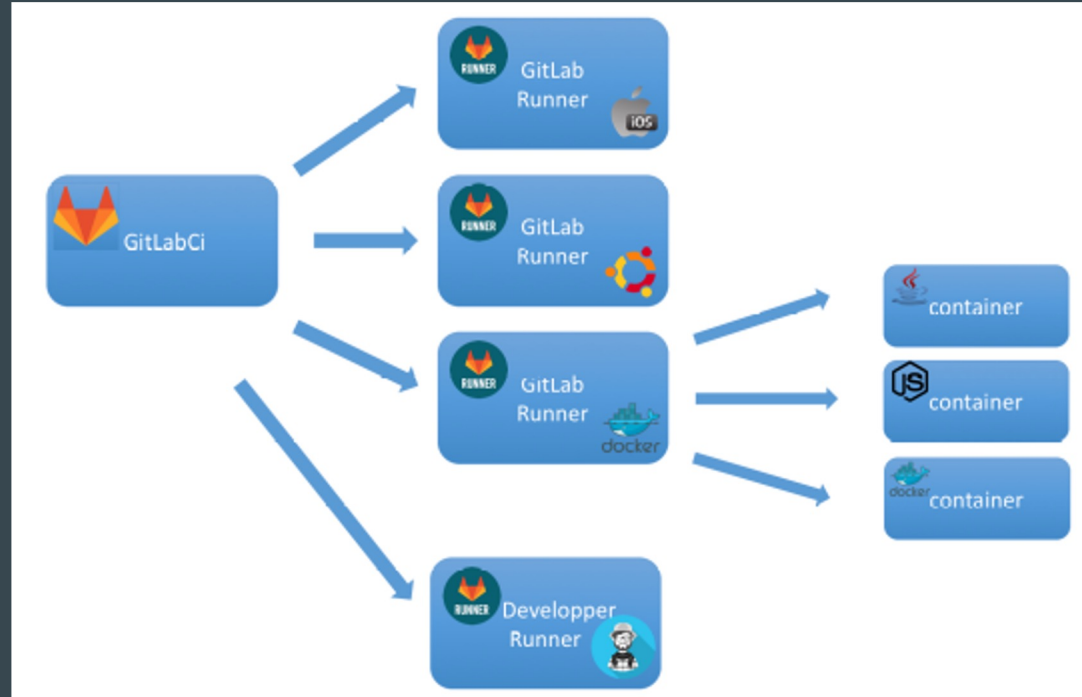
Разработка .gitlab-ci.yml

Для каждой задачи используются следующие директивы:

- `stage` — определяет стадию, к которой относится задача;
- `script` — действия, которые будут произведены, когда запустится задача;
- `when` — вид задачи (`manual` означает, что задача будет запускаться из пайплайна вручную);
- `tags` — теги, которые в свою очередь определяют, на каком раннере будет запущена задача.

Разработка .gitlab-ci.yml

Раннер — часть GitLab CI, аналогичная другим системам CI, т.е. агент, который получает задачи от GitLab и выполняет их script.



Разработка .gitlab-ci.yml

Обратите внимание на этот блок:

```
.staging-deploy: &staging-deploy
```

```
tags: [deploy]
```

```
stage: staging
```

```
when: manual
```

```
script:
```

```
- echo $CI_BUILD_NAME
```

Разработка .gitlab-ci.yml

Таким образом, задачи на стадиях build, testing, staging, pre-production, которые должны быть доступны для веток infra_, feature_ и тегов, принимают следующий вид:

```
test unit:
  stage: testing
  tags: [deploy]
  script:
  - echo "test unit"
  only:
  - tags
  - /^infra_.*$/
  - /^feature_.*$/
```

Разработка .gitlab-ci.yml

А задачи на стадиях **approve** и **production**, которые доступны только для тегов, имеют такой вид:

```
deploy to production:
```

```
stage: production
```

```
tags: [deploy]
```

```
script:
```

```
- echo "deploy to  
production!"
```

```
only:
```

```
- tags
```

Что такое Gradle?

Gradle — это система автоматизации сборки, которая является полностью открытым исходным кодом и использует концепции, которые вы видите в Apache Maven и Apache Ant.



Gradle

Что такое Maven?

Maven используется для автоматизации сборки проектов с использованием Java. Он помогает составить схему сборки конкретного программного обеспечения, а также его различных зависимостей.



Gradle в сравнении с Maven

Существуют некоторые фундаментальные различия в подходе этих двух систем к сборке. Gradle основан на графе зависимостей задач, в котором задачи – это то, что выполняет работу, в то время как Maven основан на фиксированной и линейной модели фаз. В Maven цели привязываются к фазам проекта, и цели выполняют ту же функцию, что и задачи в Gradle, являясь "элементами, которые выполняют работу".

Gradle в сравнении с Maven

Оба инструмента имеют свои сильные и слабые стороны:

- **Индивидуальные сборки.**
- **Управление зависимостями и структура каталогов.**
- **Плагины и интеграции.**
- **Гибкость.**

Спасибо за внимание !

